

Projet FEM avec l'utilisation de FEniCSx



1 Description du projet

L'objectif du projet est de réaliser un modèle 3D d'une poutre en tapering (c'est-à-dire à section non constante) et d'observer la répartition des déplacements et des contraintes. La poutre est encastree à une extrémité (celle avec la section la plus grande) et soumise à une charge à l'autre extrémité (celle avec la section la plus petite). Enfin, une comparaison sera effectuée entre différents matériaux et différentes typologies de tapering.

2 Cas initial : effilement linéaire avec de l'aluminium 7075

Le code Python réalisé est présenté intégralement dans les annexes du rapport.

2.1 Définition de la géométrie

Initialement, la géométrie de la poutre a été définie. Elle a une longueur de 12 mètres, avec une base encastree de 2 mètres de côté qui se réduit à 0,5 mètre à l'extrémité libre. Le matériau utilisé est l'aluminium 7075, largement employé dans le secteur aéronautique. Il a été décidé d'exprimer le module de Young en GPa afin de permettre une meilleure visualisation graphique des déplacements, ce qui sera utile pour les comparaisons entre les différents matériaux qui seront réalisées ultérieurement (par conséquent, les valeurs de déplacement doivent être multipliées par 10^{-9} pour obtenir la valeur correspondante avec un E exprimé en Pa).

Dans le code, delta, gamma et beta sont des paramètres géométriques utilisés pour définir ensuite le tapering. En revanche, lambda est le paramètre de Lamé.

```
L = 12.0
W = 2
W_tip = 0.5
mu = 71.7 # [GPa]
rho = 2.81
delta = W / L
gamma = 0.4 * delta**2
beta = 1.25
lambda_ = beta
g = gamma
```

2.2 Définition de la Mesh 3D

Une mesh 3D a été réalisée pour le domaine de la poutre. L'axe x a été divisé en 20 intervalles, tandis que y et z ont été divisés en 6 chacun, pour un total de 720 cellules. Les cellules sont de forme hexagonale (utiles pour des simulations structurelles 3D).

```
domain = mesh.create_box(MPI.COMM_WORLD, [np.array([0, 0, 0]), np.array([L, W, W])],
                           [20, 6, 6], cell_type=mesh.CellType.hexahedron)
```

2.3 Création de la tapering linéaire de la poutre

La tapering de la poutre a donc été réalisée. Dans ce premier cas, la tapering est linéaire et réduit la section de l'encastrement vers l'extrémité libre.

```
geometry = domain.geometry.x
for i, point in enumerate(geometry):
    taper_factor = (1 - (1 - W_tip / W) * (point[0] / L))
    geometry[i, 1] *= taper_factor
    geometry[i, 2] *= taper_factor
```

2.4 Définition de l'espace des fonctions et conditions aux limites

On procède donc à la définition de l'espace des fonctions pour le problème d'élasticité. La fonction de l'espace V est créée comme un espace de fonctions de type Lagrange, c'est-à-dire un espace de fonctions continues et dérivables, où le degré du polynôme est 1.

```
V = fem.functionspace(domain, ("Lagrange", 1, (domain.geometry.dim,)))
```

On définit un espace de fonctions de type Lagrange d'ordre un $P1$ avec une dimension du domaine en 3D. Avec celles-ci, nous pouvons approximer la déformation de la poutre.

Les conditions aux limites sont ensuite définies en imposant que le bord de la poutre où $x = 0$ soit fixé. La condition aux limites impose le vecteur de déplacement $u_D = [0, 0, 0]$ sur ce bord.

```
def clamped_boundary(x):
    return np.isclose(x[0], 0)

fdim = domain.topology.dim - 1
boundary_facets = mesh.locate_entities_boundary(domain, fdim, clamped_boundary)

u_D = np.array([0, 0, 0], dtype=default_scalar_type)
bc = fem.dirichletbc(u_D, fem.locate_dofs_topological(V, fdim, boundary_facets), V)
```

La fonction `clamped_boundary` définit la condition aux limites. En particulier, si $x_0 = 0$, le nœud correspondant est fixé à zéro.

Il en résulte donc que :

$$\mathbf{u}(x) = \mathbf{0} \quad \text{per} \quad x_0 = 0$$

2.5 Fonctions de déformation et de contrainte

La fonction de déformation $\epsilon(\mathbf{u})$ est définie comme le gradient symétrique du vecteur de déplacement $\mathbf{u}$:

```
def epsilon(u):
    return ufl.sym(ufl.grad(u))
```

La déformation symétrique est définie comme :

$$\epsilon(\mathbf{u}) = \frac{1}{2} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T)$$

La contrainte $\sigma(\mathbf{u})$ est définie en utilisant la loi de Hooke, qui inclut à la fois la composante volumétrique (avec le paramètre λ) et la composante déviatorique (avec le paramètre μ) :

```
def sigma(u):
    return lambda_ * ufl.nabla_div(u) * ufl.Identity(len(u)) + 2 * mu * epsilon(u)
```

La formule pour la tension est :

$$\sigma(\mathbf{u}) = \lambda (\nabla \cdot \mathbf{u}) \mathbf{I} + 2\mu \epsilon(\mathbf{u})$$

où $\mathbf{I}$ est la matrice identité.

2.6 Problème variationnel

Le problème variationnel consiste à trouver le vecteur de déplacements $\mathbf{u}$ qui minimise l'énergie potentielle élastique du système. La forme variationnelle du problème est :

$$a(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \sigma(\mathbf{u}) : \epsilon(\mathbf{v}) d\Omega$$

$$L(\mathbf{v}) = \int_{\Omega} f \cdot \mathbf{v} d\Omega + \int_{\partial\Omega} \mathbf{T} \cdot \mathbf{v} d\Gamma$$

où :

- $\mathbf{u}$ est le vecteur de déplacements à déterminer,
- $\mathbf{v}$ c'est une fonction test,
- f c'est la force appliquée à l'extrémité libre,
- $\mathbf{T}$ c'est la force de traction sur le bord.

Dans le code, cette formulation est exprimée comme suit :

```
u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)
f = fem.Constant(domain, default_scalar_type((0, 0, -rho * g)))
a = ufl.inner(sigma(u), epsilon(v)) * ufl.dx
L = ufl.dot(f, v) * ufl.dx + ufl.dot(T, v) * ds
```

2.7 Résolution du problème variationnel

Le problème variationnel est résolu numériquement en utilisant un solveur linéaire de type LU préconditionné :

```
problem = LinearProblem(a, L, bcs=[bc], petsc_options={"ksp_type": "preonly",
                                                       "pc_type": "lu"})
uh = problem.solve()
```

Cela résout le système linéaire pour $\mathbf{u}$, qui représente le champ de déplacements de la poutre sous l'action des forces extérieures.

2.8 Visualisation des résultats

Les dernières lignes du code sont relatives à la visualisation des résultats.

```
pyvista.start_xvfb()
p = pyvista.Plotter()
topology, cell_types, geometry = plot.vtk_mesh(V)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
```

On obtient donc le résultat du déplacement :

```
grid["u"] = uh.x.array.reshape((geometry.shape[0], 3))
actor_0 = p.add_mesh(grid, style="wireframe", color="k")
warped = grid.warp_by_vector("u", factor=1.5)
actor_1 = p.add_mesh(warped, show_edges=True)
p.show_axes()
if not pyvista.OFF_SCREEN:
    p.show()
else:
    figure_as_array = p.screenshot("deflection.png")
```

On l'exporte ensuite en XDFM :

```
with io.XDMFFile(domain.comm, "deformation.xdmf", "w") as xdmf:
    xdmf.write_mesh(domain)
    uh.name = "Deformation"
    xdmf.write_function(uh)
```

Et on obtient enfin les contraintes (après avoir calculé les contraintes de Von Mises) :

```
#Von Mises
s = sigma(uh) - 1. / 3 * ufl.tr(sigma(uh)) * ufl.Identity(len(uh))
von_Mises = ufl.sqrt(3. / 2 * ufl.inner(s, s))
V_von_mises = fem.functionspace(domain, ("DG", 0))
stress_expr = fem.Expression(von_Mises, V_von_mises.element.interpolation_points())
stresses = fem.Function(V_von_mises)
stresses.interpolate(stress_expr)

warped.cell_data["VonMises"] = stresses.x.array
warped.set_active_scalars("VonMises")

p = pyvista.Plotter()
p.add_mesh(warped)
p.show_axes()
if not pyvista.OFF_SCREEN:
    p.show()
else:
    stress_figure = p.screenshot("stresses.png")
```

3 Résultats

Nous parvenons donc à visualiser les résultats suivants :

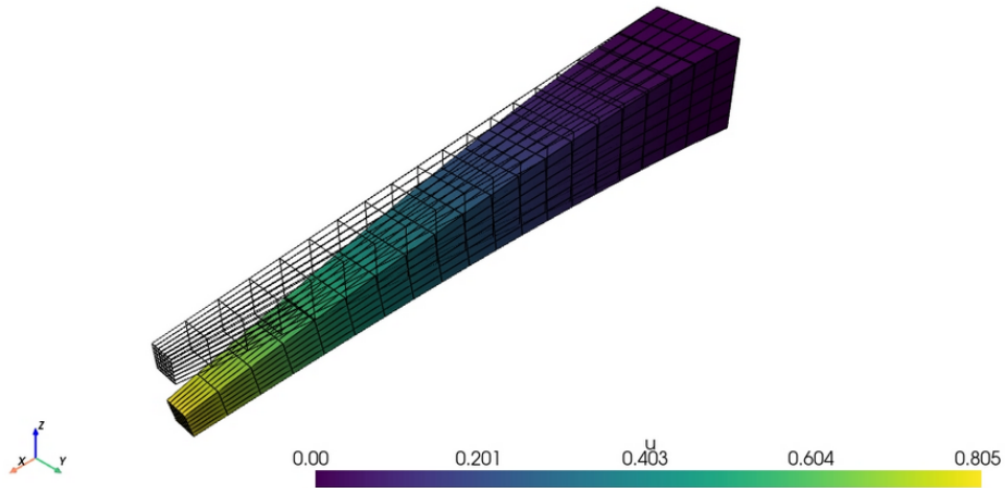


Figure 1 :Distribution du déplacement

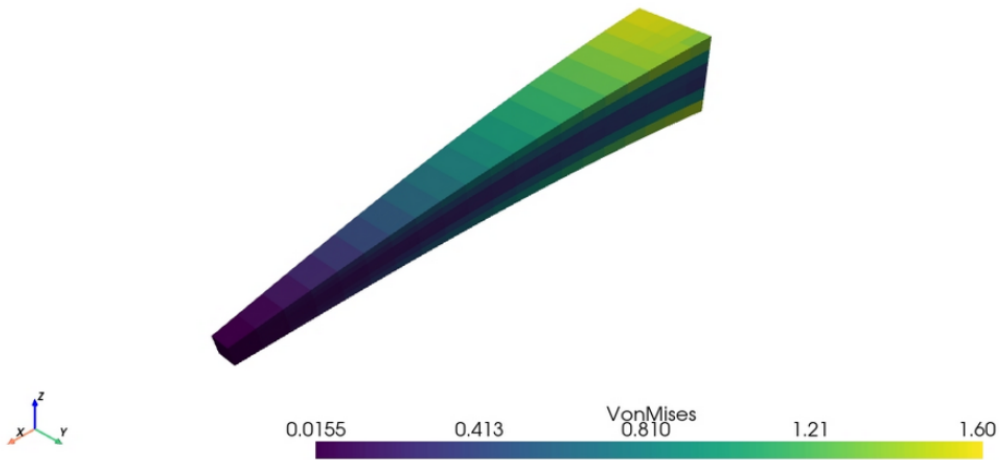


Figure 2 :Distribution des contraintes

Nous remarquons que le déplacement est nul à l'encastrement et maximal à l'extrémité libre. En revanche, le stress est maximal du côté de l'encastrement et diminue vers la pointe de la poutre. Ce comportement est intéressant d'un point de vue structurel, car cela signifie que l'extrémité libre, étant soumise à des tensions moindres, nécessite une résistance structurelle plus faible par rapport à l'extrémité encastrement. On pourrait donc envisager d'optimiser l'utilisation du matériau en réduisant la section de la poutre dans cette zone. Cela justifie ainsi l'utilisation de tapering dans des applications structurelles, comme par exemple les ailes d'avion, qui présentent une section plus petite vers la pointe, permettant une réduction du poids sans compromettre la résistance de la structure.

4 Comparaison entre différents matériaux

Ensuite, le matériau utilisé a été modifié afin d'observer le comportement de la poutre en fonction de la variation du module de Young et de la densité dans le code. Comme mentionné précédemment, le module d'élasticité est exprimé en GPa, car en l'exprimant en Pa, la flexion de la poutre devenait dans de nombreux cas visuellement imperceptible. (Cela signifie que pour obtenir des valeurs de déplacement réelles avec un E en Pa, elles devraient être multipliées par 10^{-9}).

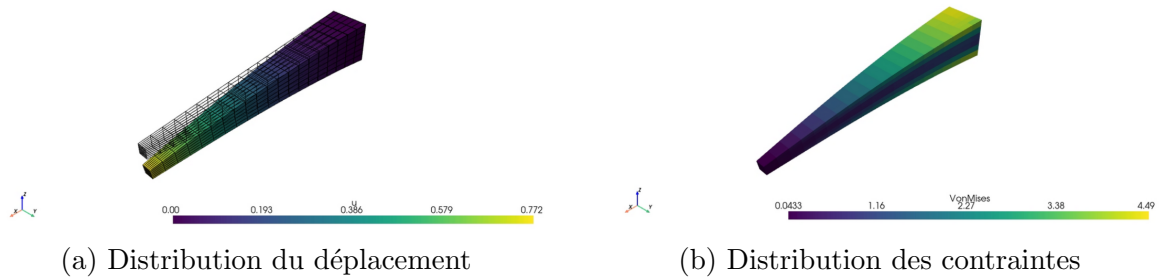


FIGURE 3 – Acier au carbone S235JR

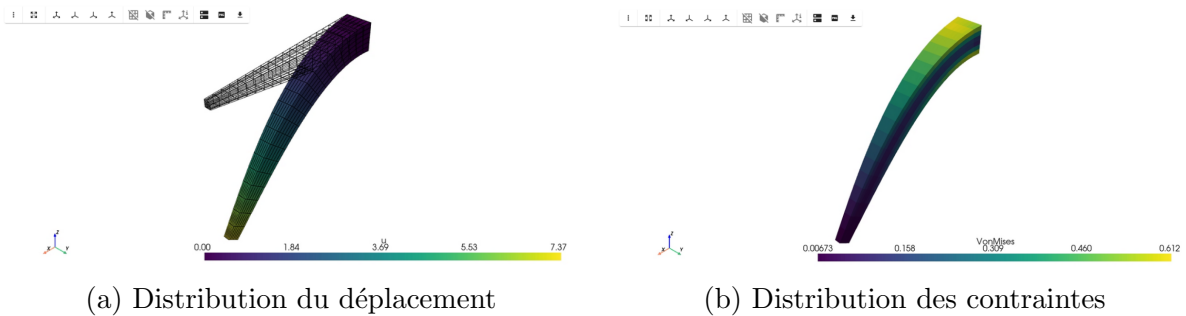


FIGURE 4 – Nylon 6/6

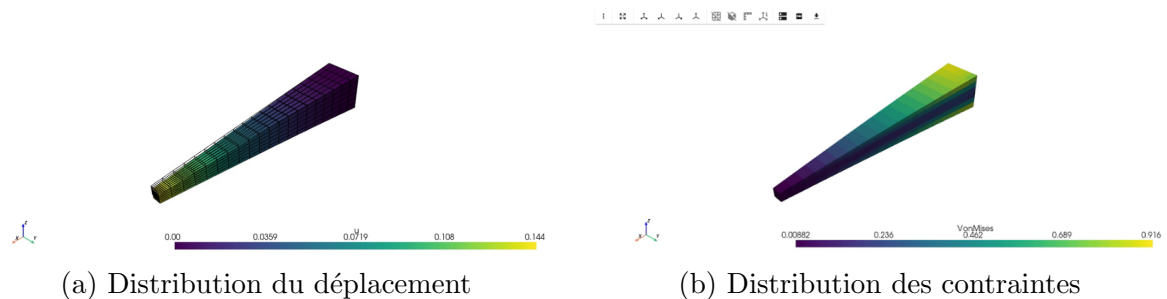


FIGURE 5 – Fibre de carbone

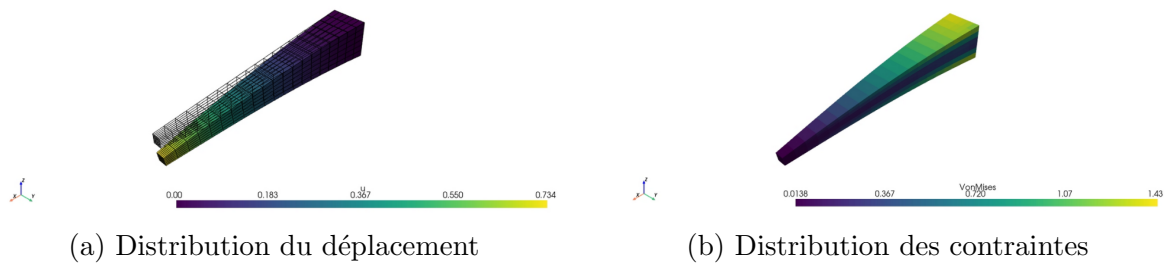


FIGURE 6 – Fibre de verre

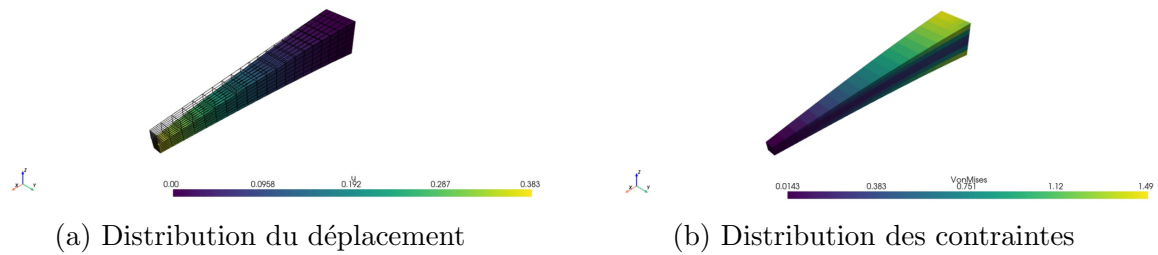


FIGURE 7 – Composé à matrice métallique (MMC), (aluminium renforcé par des fibres de carbone)

Les matériaux utilisés et leurs caractéristiques correspondantes sont résumés dans le tableau suivant :

Matériau	E [Gpa]	Densité [kg/m ³]
Al 7075	71.7	2.81
Acier au carbone S235JR	210	7.85
Nylo 6/6	2.8	1.14
Fibre de carbone	230	1.6
Fibre de verre	70	2.5
MMC	140	2.6

Figure 8 :Matériaux utilisés

Les matériaux montrent des comportements différents en fonction de leurs modules de Young et de leur densité. Les matériaux avec des modules d'élasticité plus élevés, comme l'acier au carbone et la fibre de carbone, ont tendance à présenter des déformations (déplacements) plus faibles, car ils sont plus rigides. En revanche, les matériaux avec des modules de Young plus faibles, comme le Nylon 6/6, sont plus déformables. Cependant, dans le cadre d'une étude en aéronautique, la légèreté structurelle joue un rôle important. Il faut donc garantir à la fois la rigidité et réduire le poids de la structure. Par exemple, la fibre de carbone, avec un E élevé et une densité relativement faible, offre un bon compromis entre rigidité et poids par rapport à l'aluminium 7075, présentant à la fois un déplacement et une contrainte inférieurs.

5 Exemples de tapering non uniformes

Tapering quadratique : tapering qui varie de manière quadratique le long de la longueur de la poutre. La loi a été exprimée dans le code comme suit :

```
taper_factor = (1 - (1 - W_tip / W) * (point[0] / L)**2)
```

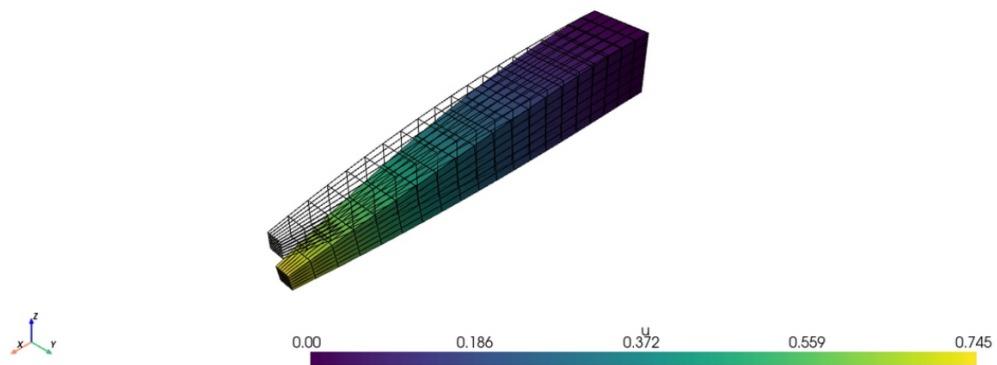


Figure 9 :Distribution du déplacement

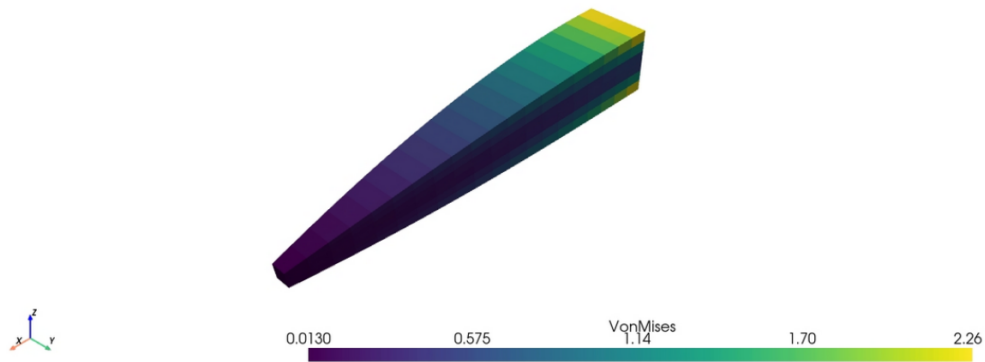


Figure 10 :Distribution des contraintes

Tapering exponentiel : tapering qui varie selon une courbe exponentielle le long de la longueur de la poutre.

```
alpha=0.1  
taper_factor = np.exp(-alpha * point[0])
```

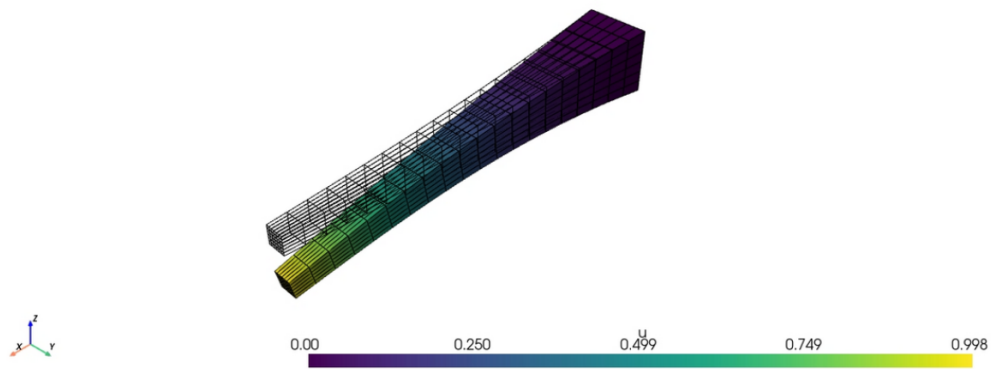


Figure 11 :Distribution du déplacement

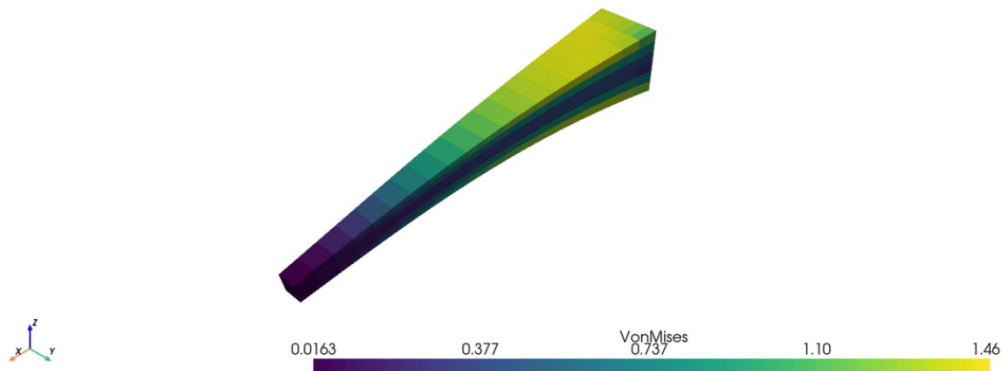


Figure 12 :Distribution des contraintes

Tapering arbitraire qui varie de manière non linéaire :

```
n=1
taper_factor = 1 - ((point[0] / L)**n)
```

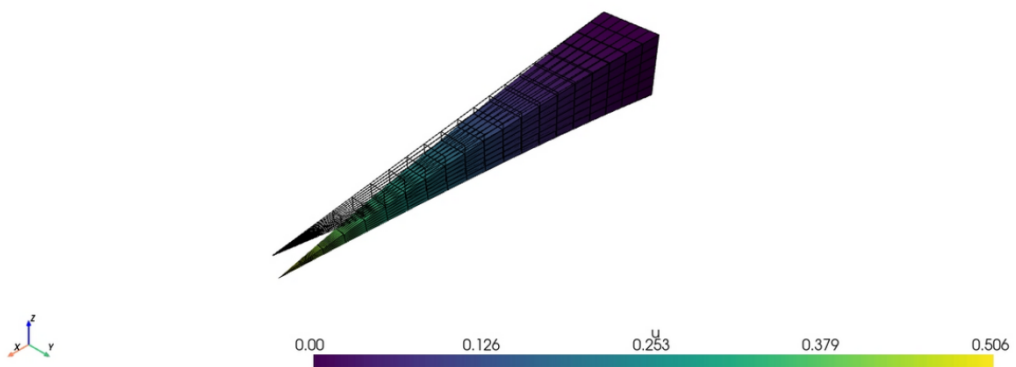


Figure 13 :Distribution du déplacement

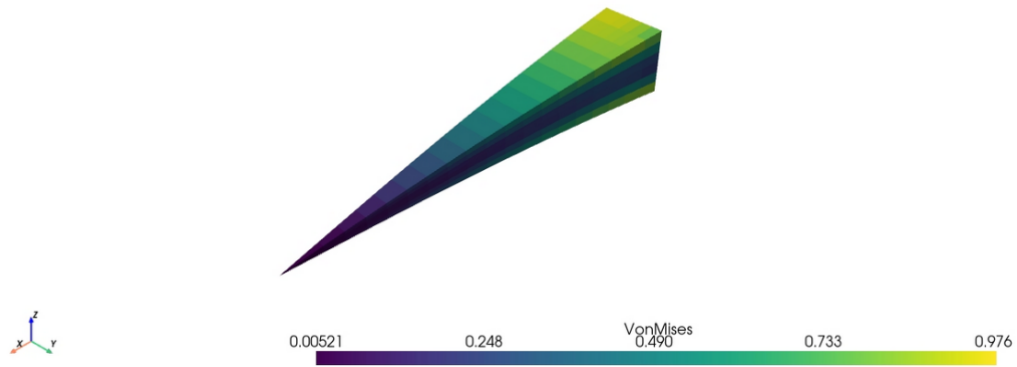


Figure 14 :Distribution des contraintes

Selon la façon dont les taperings ont été réalisés dans cette analyse, en se basant sur les mêmes dimensions initiales et le même matériau (aluminium 7075), il apparaît que le tapering non linéaire arbitraire (le troisième réalisé) présente à la fois un déplacement et une contrainte inférieurs par rapport aux autres.

6 Annexes

```
import pyvista
from dolfinx import mesh, fem, plot, io, default_scalar_type
from dolfinx.fem.petsc import LinearProblem
from mpi4py import MPI
import ufl
import numpy as np

# Dati trave
L = 12.0 # Lunghezza della trave
W = 2 # Larghezza base
W_tip = 0.5 # Larghezza punta
mu = 71.7 # Modulo di elasticità [GPa]
rho = 2.81 # Densità
delta = W / L
gamma = 0.4 * delta**2
beta = 1.25
lambda_ = beta
g = gamma

# Mesh
domain = mesh.create_box(MPI.COMM_WORLD, [np.array([0, 0, 0]), np.array([L, W, W])],
                          [20, 6, 6], cell_type=mesh.CellType.hexahedron)

# Rastremare la mesh lungo l'asse x
geometry = domain.geometry.x
for i, point in enumerate(geometry):
    taper_factor = (1 - (1 - W_tip / W) * (point[0] / L)) # Fattore di
                                                         rastremazione
    geometry[i, 1] *= taper_factor # Scala la coordinata y
    geometry[i, 2] *= taper_factor # Scala la coordinata z

# Funzione spazio e condizioni al contorno
V = fem.functionspace(domain, ("Lagrange", 1, (domain.geometry.dim,)))

def clamped_boundary(x):
    return np.isclose(x[0], 0)

fdim = domain.topology.dim - 1
boundary_facets = mesh.locate_entities_boundary(domain, fdim, clamped_boundary)

u_D = np.array([0, 0, 0], dtype=default_scalar_type)
bc = fem.dirichletbc(u_D, fem.locate_dofs_topological(V, fdim, boundary_facets), V)
T = fem.Constant(domain, default_scalar_type((0, 0, 0)))
ds = ufl.Measure("ds", domain=domain)
```

```

# Funzioni di deformazione e tensione
def epsilon(u):
    return ufl.sym(ufl.grad(u))

def sigma(u):
    return lambda_ * ufl.nabla_div(u) * ufl.Identity(len(u)) + 2 * mu * epsilon(u)

# Problema variazionale
u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)
f = fem.Constant(domain, default_scalar_type((0, 0, -rho * g)))
a = ufl.inner(sigma(u), epsilon(v)) * ufl.dx
L = ufl.dot(f, v) * ufl.dx + ufl.dot(T, v) * ds

# Risoluzione del problema
problem = LinearProblem(a, L, bcs=[bc], petsc_options={"ksp_type": "preonly",
                                                        "pc_type": "lu"})
uh = problem.solve()

# Visualizzazione dei risultati
pyvista.start_xvfb()
p = pyvista.Plotter()
topology, cell_types, geometry = plot.vtk_mesh(V)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)

# Deformazione
grid["u"] = uh.x.array.reshape((geometry.shape[0], 3))
actor_0 = p.add_mesh(grid, style="wireframe", color="k")
warped = grid.warp_by_vector("u", factor=1.5)
actor_1 = p.add_mesh(warped, show_edges=True)
p.show_axes()
if not pyvista.OFF_SCREEN:
    p.show()
else:
    figure_as_array = p.screenshot("deflection.png")

# Esportazione in XDMF
with io.XDMFFile(domain.comm, "deformation.xdmf", "w") as xdmf:
    xdmf.write_mesh(domain)
    uh.name = "Deformation"
    xdmf.write_function(uh)

# Calcolo delle tensioni di Von Mises
s = sigma(uh) - 1. / 3 * ufl.tr(sigma(uh)) * ufl.Identity(len(uh))
von_Mises = ufl.sqrt(3. / 2 * ufl.inner(s, s))

```

```

V_von_mises = fem.functionspace(domain, ("DG", 0))
stress_expr = fem.Expression(von_Mises, V_von_mises.element.interpolation_points())
stresses = fem.Function(V_von_mises)
stresses.interpolate(stress_expr)

# Visualizzazione delle tensioni di Von Mises
warped.cell_data["VonMises"] = stresses.x.array
warped.set_active_scalars("VonMises")

p = pyvista.Plotter()
p.add_mesh(warped)
p.show_axes()
if not pyvista.OFF_SCREEN:
    p.show()
else:
    stress_figure = p.screenshot("stresses.png")

```